

FÍSICA PARA JOGOS DIGITAIS · GRAU A

Motores de Física nas Game Engines Modernas

Um levantamento do estado da técnica

Unity · **Unreal Engine** · **Godot**

Docente: Prof.^a Rossana Baptista Queiroz

Acadêmico: Matheus Dubin da Silveira

Universidade do Vale do Rio dos Sinos · Escola Politécnica · 2026

01

Introdução

Por que revisar a física dos motores agora?

Historicamente, a física dos motores dependia de bibliotecas de terceiros (**PhysX** antigo, **Havok**) focadas quase só em **corpos rígidos**.

- As três grandes engines passaram por **reformulações arquitetônicas profundas** em seus sistemas de física.
- Surgem soluções de **larga escala, processamento paralelo** e **simulações complexas** — destruição, tecidos, fluidos.
- **Objetivo:** levantar o estado da técnica e comparar como cada engine lida com problemas modernos de simulação em tempo real.

As três engines em análise

	Unity 6 (LTS)	Unreal Engine 5.8	Godot 4.6
Perfil	Ecossistema maduro; forte em mobile/indie/AA e simulação determinística	Referência AAA em fidelidade visual e efeitos de física	Código aberto, leve, adoção crescente
Motor de física padrão	NVIDIA PhysX 4.x	Chaos Physics (Epic)	Jolt Physics (código aberto)

Versões de referência congeladas em 2026 para garantir comparação justa.

02

Panorama Atual

Motores subjacentes de cada engine

	Unity 6	Unreal 5.8	Godot 4.6
Motor padrão	NVIDIA PhysX 4.x	Chaos Physics (proprietário)	Jolt Physics (código aberto)
Alternativa oficial	Unity Physics (DOTS/ECS)	— (Chaos é único)	Godot Physics (legado)
Descontinuada	Havok for Unity (removido do Pro, 2025)	PhysX (deprecated em UE 5.1)	—
Solver base	Impulse-based (PhysX) / XPBD (Unity Physics)	XPBD (Extended Position-Based Dynamics)	Impulse-based (Jolt)

Tendência comum: saída das libs de terceiros rumo a motores próprios/abertos e solvers baseados em posição.

Como um motor de física funciona (a cada passo)

Vocabulário que usaremos: todo motor repete o mesmo pipeline a cada passo fixo de simulação. Saber onde cada etapa gasta tempo é o que torna a comparação — e os benchmarks do Grau B — legíveis.

- **1 · Broadphase** — descobre rapidamente quais pares podem colidir (estruturas como BVH / SAP).
- **2 · Narrowphase** — testa a geometria exata desses pares (algoritmos GJK / EPA).
- **3 · Solver** — calcula e aplica correções para resolver contatos e juntas, iterando. **É o coração e o maior custo.**
- **4 · Integração** — atualiza velocidades e posições para o próximo passo.
- **5 · Sleep** — corpos praticamente parados "dormem" para poupar CPU.

Por que importa: é sobretudo no **solver** que as engines divergem — e é o Physics Step Time desse estágio que mediremos no Grau B.

O coração: os tipos de solver

O **solver** decide como resolver contatos e juntas a cada passo. Há duas grandes famílias — e é isso que separa as três engines:

Família	Como funciona	Quem usa
Baseado em impulso (Gauss-Seidel iterativo)	Corrige velocidades par a par, iterando. No PGS (padrão do Unity) a posição só é corrigida no fim; no TGS (opt-in) ela é atualizada durante as iterações → mais estável em pilhas e cadeias.	PhysX (Unity) · Jolt (Godot)
Baseado em posição (PBD / XPBD)	Resolve diretamente as posições para satisfazer as restrições; o XPBD adiciona rigidez fisicamente correta.	Chaos (Unreal) · Unity Physics (DOTS)

Regra de ouro: mais iterações/substeps = mais estável, porém mais caro. **Nenhum sustenta pilhas altas com os valores padrão** — algo que o Grau B confirma na prática.

03

Análise Comparativa

Colisão: discreta vs. contínua

O problema: a detecção **discreta** testa a sobreposição uma vez por passo — barata, mas um objeto veloz pode **atravessar** uma parede fina entre dois passos (tunneling). A **contínua (CCD)** varre a trajetória para impedir isso — precisa, porém mais cara. As três usam discreta por padrão, com CCD opcional.

	Unity	Unreal	Godot
Como resolve	<code>Rigidbody</code> + <code>Collider</code> (PhysX). CCD por flag <code>Continuous</code>	Chaos <code>FPBDRigidsSolver</code> . CCD por componente; dupla precisão (LWC)	<code>RigidBody3D</code> . Jolt: speculative contacts + motion clamping

A diferença que importa: mesma escolha padrão (discreta); divergem no truque de precisão — os speculative contacts do Jolt e a dupla precisão (LWC) da Unreal para mundos de escala quilométrica.

Juntas: coordenadas máximas vs. reduzidas

O **conceito**: juntas restringem como dois corpos se movem entre si (dobradiça = porta, esfera = ombro). Cadeias articuladas podem usar **coordenadas máximas** (cada corpo é livre e a restrição o "puxa" — pode vibrar/deslizar) ou **reduzidas** (só os graus de liberdade permitidos existem — estável por construção; ideal p/ robótica e ragdolls).

	Unity	Unreal	Godot
Tipos & abordagem	Hinge/Fixed/Spring/Configurable (6 DoF) + ArticulationBody (coord. reduzidas)	PhysicsConstraint único: ball/hinge/prismático; motors; Breakable; profiles em runtime	Pin / Hinge / Slider / ConeTwist / Generic6DOF (coord. máximas)

A **diferença que importa**: só a Unity oferece **coordenadas reduzidas** (ArticulationBody) para cadeias estáveis; a Unreal aposta em ajuste em runtime + blend com animação; a Godot cobre o conjunto padrão.

Soft bodies: tecido vs. volumétrico

Dois níveis de "mole": **tecido (cloth)** é uma **superfície 2D** — malha de restrições de aresta (bandeiras, capas). **Soft body volumétrico** é um **sólido 3D** que se deforma por dentro (malha tetraedral) — músculo, gelatina. O volumétrico é muito mais caro e difícil.

	Unity	Unreal	Godot
Suporte	<code>Cloth</code> (superfície). Sem volumétrico nativo	Chaos Cloth (PBD, CPU) + Chaos Flesh (volumétrico tetraedral, experimental)	<code>SoftBody3D</code> (tecido). Não volumétrico ; bugs com Jolt

A diferença que importa: as três fazem tecido; só a Unreal tenta volumétrico nativo (Chaos Flesh) — e ainda experimental. **Nenhuma tem soft body volumétrico pronto para produção.**

Destruição: pré-fraturada vs. procedural

Duas abordagens: pré-fraturada — o artista quebra a malha antes e troca no impacto (barato, "enlatado"). **Procedural em tempo real** — a engine fratura dinamicamente (células de Voronoi) e reage às forças (cinematográfico, caro).

	Unity	Unreal	Godot
Suporte	Sem sistema nativo — Asset Store (DinoFracture, Blast)	Chaos Destruction + Geometry Collections (Voronoi/Slice), tempo real; Field System. Produção 5.4+	Sem sistema nativo — addons + peças pré-quebradas

A diferença que importa: a Unreal é a **única com destruição procedural AAA nativa**; Unity e Godot dependem de pré-fratura ou addons comunitários.

Fluidos: partícula visual vs. solver físico

A **distinção-chave: partículas visuais** parecem fluido mas não empurram sólidos de volta. Um **solver físico** (SPH/FLIP) simula o líquido de verdade. E há **acoplamento**: uma via (o fluido só recebe) vs. duas vias (fluido e sólidos se empurram mutuamente).

	Unity	Unreal	Godot
Suporte	VFX Graph (GPU) + Particle System. Sem solver de fluidos nativo	Niagara Fluids (GPU FLIP, produção; colide via SDF) + Chaos Fluids (duas vias, experimental)	<code>GPUParticles3D</code> + colisão. Sem solver dedicado

A **diferença que importa**: a Unreal lidera (Niagara em produção; acoplamento de duas vias em desenvolvimento); Unity e Godot oferecem partículas visuais, não fluidos físicos.

Arquitetura: onde a física roda e como escala

Ideias-chave: **threads** (1 vs. várias); **ECS/DOTS** = dados orientados, stateless → **determinístico** (mesma entrada = mesma saída, essencial p/ rollback em rede); **islands** = grupos independentes resolvidos em paralelo; **Async Tick** = física num relógio próprio, desacoplada do FPS. **Nenhuma usa GPU para corpos rígidos.**

	Unity	Unreal	Godot
Modelo	PhysX: worker thread (stateful). DOTS: ECS + Job System + Burst — stateless/determinístico	<code>FPhysScene_Chaos</code> : islands paralelas + Async Physics Tick (5.4+) + LWC	GodotPhysics: 1 thread. Jolt: multi-thread nativo (todos os cores); <code>PhysicsServer3D</code> trocável

A diferença que importa: Unity = determinismo (DOTS); Unreal = async + islands + mundos gigantes; Godot = multithread aberto e trocável. Todas presas à **CPU** para corpos rígidos.

Pontos fortes e fracos

Unity 6

- ▲ Física determinística (DOTS), **ArticulationBody** p/ robótica, ecossistema maduro
- ▼ Destruição nativa, soft bodies volumétricos, fluidos físicos

Unreal 5.8

- ▲ Destruição AAA (Chaos), fluidos visuais (Niagara), ragdolls complexos
- ▼ Chaos Flesh/Fluids ainda experimentais, Cloth em CPU

Godot 4.6

- ▲ Código aberto, multi-threading (Jolt), GDExtension, custo zero
- ▼ Destruição nativa, fluidos físicos, SoftBody limitado

Nenhuma domina em tudo: cada engine otimizou para um perfil de uso diferente.

04

Considerações Finais

Para onde a simulação física caminha

- **Convergência de solvers:** as três caminham para **XPBD/PBD**, maior **multi-threading** e GPU reservada a efeitos específicos (fluidos, partículas).
- **Unity** — a única com ECS **stateless e determinístico**: vantagem para jogos online competitivos.
- **Unreal** — a única com **destruição procedural AAA nativa** e acoplamento fluido-sólido em desenvolvimento.
- **Godot** — a única **de código aberto**: pipeline de física customizável via GDExtension, sem custo de licença.

PONTE PARA O GRAU B

Da teoria à prova

No **Grau B** colocamos estas arquiteturas à prova com benchmarks idênticos nas três engines — medindo estabilidade de pilha e estresse de colisão.

Cenário 1 · A Torre — estabilidade & jitter

Cenário 2 · A Chuva — 1k / 5k / 10k corpos

A pergunta que responderemos: **os resultados confirmam o que a arquitetura previu?**

05

Referências

Fontes primárias por engine

- **Unity** — Built-in 3D Physics Overview; Unity Physics (com.unity.physics); DOTS / Burst / C# Job System. docs.unity3d.com
- **Unreal** — Physics in Unreal Engine; Chaos Cloth / Flesh; Chaos Destruction & Geometry Collections; Niagara Fluids; Large World Coordinates. dev.epicgames.com
- **Godot** — Using Jolt Physics (4.6); RigidBody3D / Joint3D / SoftBody3D; PhysicsServer3D; godot-jolt (GitHub). docs.godotengine.org

GDC · SIGGRAPH · arXiv · SBGames

- Rouwé, J. — **Architecting Jolt Physics for 'Horizon Forbidden West'**, GDC 2022.
- Macklin, M. et al. — **Small Steps in Physics Simulation**, SIGGRAPH/SCA 2019 (base do XPBD com substeps).
- NVIDIA — **PhysX 4: Raising the Fidelity & Performance**, GTC 2019 (solver TGS).
- Van Allen, J. et al. (Epic) — **Causing Chaos: The Future of Physics and Destruction in UE**, GDC 2019.
- Balakshin, A. — **ECS in Practice: The Case of Unity**, GDC 2024.
- Kaup, M. et al. — **A Review of Nine Physics Engines for RL Research**, arXiv 2024.
- Oliveira, S. et al. — **Decoupling Physics from the Game Engine**, SBGames 2023.

Obrigado

Física para Jogos Digitais

Docente: Prof.^a Rossana Baptista Queiroz

Acadêmico: Matheus Dubin da Silveira

Continua no Grau B → benchmarks práticos

